```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/20/2021 03:58:58 PM
// Design Name:
// Module Name: top_level
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module top_level(
    input wire [15:0] sw,
    input wire btnd,
    input wire btnc,
    input wire btnu,
    input wire btnl,
    input wire clk_100mhz,
    output logic ca, cb, cc, cd, ce, cf, cg,
    output logic [7:0] an
    );

    logic [6:0] temp_cat_out;
    logic clean;

    logic [15:0] sw_10hz;
    logic [15:0] sw_count;
    logic [15:0] sw_out;

    logic old_clean;
    logic rising_clean;

    parameter PERIOD = 32'd10_000_000;
    logic[31:0] periodic_counter; //32 bits is more than sufficient!
    logic pulse;

    assign rising_clean = clean & !old_clean;0

    always_ff @(posedge clk_100mhz) begin
        if (btnd) begin
            old_clean <= 1'b0;
        end else begin
            old_clean <= clean;
        end
    end



    assign pulse = (periodic_counter == PERIOD-1);

    always_ff @(posedge clk_100mhz) begin
        if (btnd) begin
            periodic_counter <= 32'b0;
        end else begin
```

```
            if (periodic_counter == PERIOD-1)begin
                periodic_counter <= 32'b0;
            end else begin
                periodic_counter <= periodic_counter + 32'b1;
            end
        end
    end

    debounce deb (.clk_in(clk_100mhz), .rst_in(btnd), .bouncey_in(btnu), .clean_out(cl
ean));

    simple_counter sc1(.clk_in(clk_100mhz), .rst_in(btnd), .evt_in(pulse), .count_out(
sw_10hz));

    simple_counter sc2(.clk_in(clk_100mhz), .rst_in(btnd), .evt_in(rising_clean), .cou
nt_out(sw_count));

    multiplexer mu(.in_sw(sw), .in_count(sw_count), .in_10hz(sw_10hz), .in_btnc(btnc),
 .in_btnl(btnl), .out_val(sw_out));


    seven_seg_controller ssc(.val_in(sw_out), .rst_in(btnd), .clk_in(clk_100mhz), .cat
_out(temp_cat_out), .an_out(an));
    assign ca = temp_cat_out[0];
    assign cb = temp_cat_out[1];
    assign cc = temp_cat_out[2];
    assign cd = temp_cat_out[3];
    assign ce = temp_cat_out[4];
    assign cf = temp_cat_out[5];
    assign cg = temp_cat_out[6];

endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/20/2021 04:08:49 PM
// Design Name:
// Module Name: seven_seg_controller
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


`default_nettype none

module seven_seg_controller(input wire         clk_in,
                            input wire         rst_in,
                            input wire [31:0]  val_in,
                            output logic [6:0]   cat_out,
                            output logic [7:0]   an_out
    );

    logic [7:0]     segment_state;
```

```
    logic [31:0]    segment_counter;
    logic [3:0]     routed_vals;
    logic [6:0]     led_out;

    binary_to_seven_seg my_converter ( .bin_in(routed_vals), .hex_out(led_out));
    assign cat_out = ~led_out;
    assign an_out = ~segment_state;


    always_comb begin
        case(segment_state)
            8'b0000_0001:   routed_vals = val_in[3:0];
            8'b0000_0010:   routed_vals = val_in[7:4];
            8'b0000_0100:   routed_vals = val_in[11:8];
            8'b0000_1000:   routed_vals = val_in[15:12];
            8'b0001_0000:   routed_vals = val_in[19:16];
            8'b0010_0000:   routed_vals = val_in[23:20];
            8'b0100_0000:   routed_vals = val_in[27:24];
            8'b1000_0000:   routed_vals = val_in[31:28];
            default:        routed_vals = val_in[3:0];
        endcase
    end

    always_ff @(posedge clk_in)begin
        if (rst_in)begin
            segment_state <= 8'b0000_0001;
            segment_counter <= 32'b0;
        end else begin
            if (segment_counter == 32'd100_000)begin
                segment_counter <= 32'd0;
                segment_state <= {segment_state[6:0],segment_state[7]};
            end else begin
                segment_counter <= segment_counter +1;
            end
        end
    end

endmodule //seven_seg_controller


//feel free to either include binary_to_seven_seg module here or in its own file!
module binary_to_seven_seg (
        input wire [3:0]        bin_in,  //declaring input explicitely
        output logic [6:0]      hex_out);  //declaring output explicitely


        always @(bin_in) begin
            case (bin_in)
                4'b0000 : hex_out = 7'h3F;
                4'b0001 : hex_out = 7'h06;
                4'b0010 : hex_out = 7'h5B;
                4'b0011 : hex_out = 7'h4F;

                4'b0100 : hex_out = 7'h66;
                4'b0101 : hex_out = 7'h6D;
                4'b0110 : hex_out = 7'h7D;
                4'b0111 : hex_out = 7'h07;

                4'b1000 : hex_out = 7'h7F;
                4'b1001 : hex_out = 7'h67;
                4'b1010 : hex_out = 7'h77;
                4'b1011 : hex_out = 7'h7C;

                4'b1100 : hex_out = 7'h39;
                4'b1101 : hex_out = 7'h5E;
```

```
                4'b1110 : hex_out = 7'h79;
                4'b1111 : hex_out = 7'h71;
            endcase
        end
endmodule // binary_to_seven
`default_nettype wire


`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/20/2021 05:27:30 PM
// Design Name:
// Module Name: simple_counter
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


`default_nettype none

module simple_counter(  input wire          clk_in,
                        input wire          rst_in,
                        input wire          evt_in,
                        output logic[15:0]  count_out
    );

    always_ff @(posedge clk_in)begin
        if(rst_in)begin
            count_out <= 16'b0; //reset signal
        end else begin
            //your code here
            if(evt_in) begin
                count_out <= count_out + 1;
            end
        end
    end
endmodule
`default_nettype wire


`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/20/2021 10:07:28 PM
// Design Name:
// Module Name: debounce
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
```

```
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////

`default_nettype none
module debounce(    input  wire     clk_in, //clock in
                    input  wire     rst_in, //reset in
                    input  wire     bouncey_in,//raw input to the system
                    output logic    clean_out //debounced output
    );
   logic [19:0] count; // is 20 bits enough?
   logic old;

   always_ff @(posedge clk_in) begin
        //Your Verilog here!
        if(rst_in) begin
            clean_out <= 0;
        end else begin
            if(old == bouncey_in) begin
                count <= count+1;
                if(count > 20'd999_999) begin
                    clean_out <= old;
                end
            end else begin
                count <= 0;
            end
            old <= bouncey_in;
         end
   end

endmodule
`default_nettype wire

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/21/2021 07:30:10 PM
// Design Name:
// Module Name: multiplexer
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////

`default_nettype none
module multiplexer(
    input wire [15:0] in_sw,
    input wire [15:0] in_count,
    input wire [15:0] in_10hz,
    input wire in_btnc,
    input wire in_btnl,
```

```
    output logic [15:0] out_val
    );
    always_comb begin
        if(in_btnl) begin
            out_val  = in_count;
        end else begin
            if(in_btnc) begin
                out_val = in_10hz;
            end else begin
                out_val = in_sw;
            end

        end

    end
endmodule
`default_nettype wire


UART STUFF:

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/21/2021 09:22:20 PM
// Design Name:
// Module Name: top_level
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////

`default_nettype none

module top_level(   input wire              clk_100mhz,
                    input wire [15:0]       sw,
                    input wire              btnc,
                    input wire              btnd,
                    output logic [15:0]     led,
                    output logic            uart_rxd_out
    );

    logic               clean;
    logic               old_clean;
    assign led = sw;

    always_ff @(posedge clk_100mhz)begin
        old_clean <= clean;  //for rising edge detection
    end

    debounce my_deb(.clk_in(clk_100mhz),
                    .rst_in(btnd),
                    .bouncey_in(btnc),
                    .clean_out(clean));
```

```
    serial_tx my_tx(.clk_in(clk_100mhz),
                    .rst_in(btnd),
                    .trigger_in(clean&~old_clean),
                    .val_in(sw[7:0]),
                    .data_out(uart_rxd_out));
endmodule//top_level


`default_nettype wire

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/21/2021 09:35:58 PM
// Design Name:
// Module Name: serial_tx
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////

`default_nettype none
module serial_tx(   input wire      clk_in,
                    input wire      rst_in,
                    input wire      trigger_in,
                    input wire [7:0]    val_in,
                    output logic    data_out);
    parameter   DIVISOR = 868; //treat this like a constant!!

    logic [9:0]         shift_buffer; //10 bits...interesting
    logic [31:0]        time_count;

    always_ff @(posedge clk_in) begin
        if(rst_in) begin
            shift_buffer <= 10'b0;
            time_count <= 32'b0;
            data_out <= 1'b1;
        end else begin
            if(trigger_in) begin
                shift_buffer = {1'b1, val_in, 1'b0};
                time_count <= 32'b0;
            end
            if(time_count >= DIVISOR - 1) begin
                time_count <= 32'b0;
                data_out <= shift_buffer[0];
                shift_buffer <= {1'b1, shift_buffer[9:1]};
            end else begin
                time_count <= time_count + 32'b1;
            end
        end
    end
endmodule
`default_nettype wire
```

~